```
"""Manage the order of events along the simulation timeline.

Model elements register with the SimTime object to be notified - obj.tick() -
every cycle.  The notification is organized in multiple phases (for objects in
the same phase the notification order depends on their registration order,
making the semantics unpredicatable if they are directly connected).

"""

from sets import Set

class SimTime(object):

    """Advance time, notifying interested objects every cycle in phases."""

    def __init__(self, phases):
        """`phases` should be a sequence of classes."""
        self.t = 0
        self.phases = {}
        for p in phases:
            self.phases[p] = Set()

    def register(self, obj):
        """Add obj to corresponding phase of notification"""
        phase = None
        for p in self.phases:
            if isinstance(obj, p):
                if not phase or issubclass(p, phase): # most specific wins
                    phase = p
        if phase:
            self.phases[phase].add(obj)
        else:
            raise Exception("Bogus phase")

    def unregister(self, obj):
        """Remove obj from his phase."""
        for phase in self.phases.itervalues():
            phase.discard(obj)

    def advance(self, dt, progress=100):
        for i in range(dt):
            self.t += 1
            if self.t % progress == 0:
                print self.t
            for phase in self.phases.itervalues():
                # Must make copy of phase because cars can be added
                # dynamically (actually not in the Car phase so this should
                # work anyway; let's not take the risk).
                for obj in list(phase):
                    obj.tick()


class Timed(object):

    """An object that wants to be notified every cycle

    Note that registration is performed by the wrapped classes created by a
    city, not by the object itself (and then also self.simtime is set to the
    simtime object).

    """

    def retire(self):
        """Unregister from simtime to allow recycling"""
        if hasattr(self, 'city'):
            self.city.unregister(self)

    def tick(self):
        """Called by SimTime every cycle; subclasses should implement this"""
        pass
```